

p1

Exported 3/12/2026, 4:36:23 PM

1.1

ASE3796 HW08

1. $Y = f(X) = X^3$ $X \sim N(1, \sigma^2)$; $\bar{X} = 1$, $\text{var}(X) = \sigma^2 = V$

$E[f(X)] = f(\bar{X}) = 1^3 = 1$

$\text{var}(f(X)) = \nabla f(\bar{X}) \nabla f(\bar{X})^T = 3\sigma^2 \cdot 3$

$\nabla f(X) = 3X^2$
 $\nabla f(\bar{X}) = 3$

$\text{var}(f(X)) = 9\sigma^2$

if $\sigma^2 = 0.01$, $\begin{cases} E(Y) = 1 \\ \text{var}(Y) = 9 \times 10^{-2} \end{cases}$

if $\sigma^2 = 0.1$, $\begin{cases} E(Y) = 1 \\ \text{var}(Y) = 9 \times 10^{-1} \end{cases}$

1.2

In [5]:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
def f(sigma, M):

    X = np.random.normal(1, sigma, M)
    Y = X**3

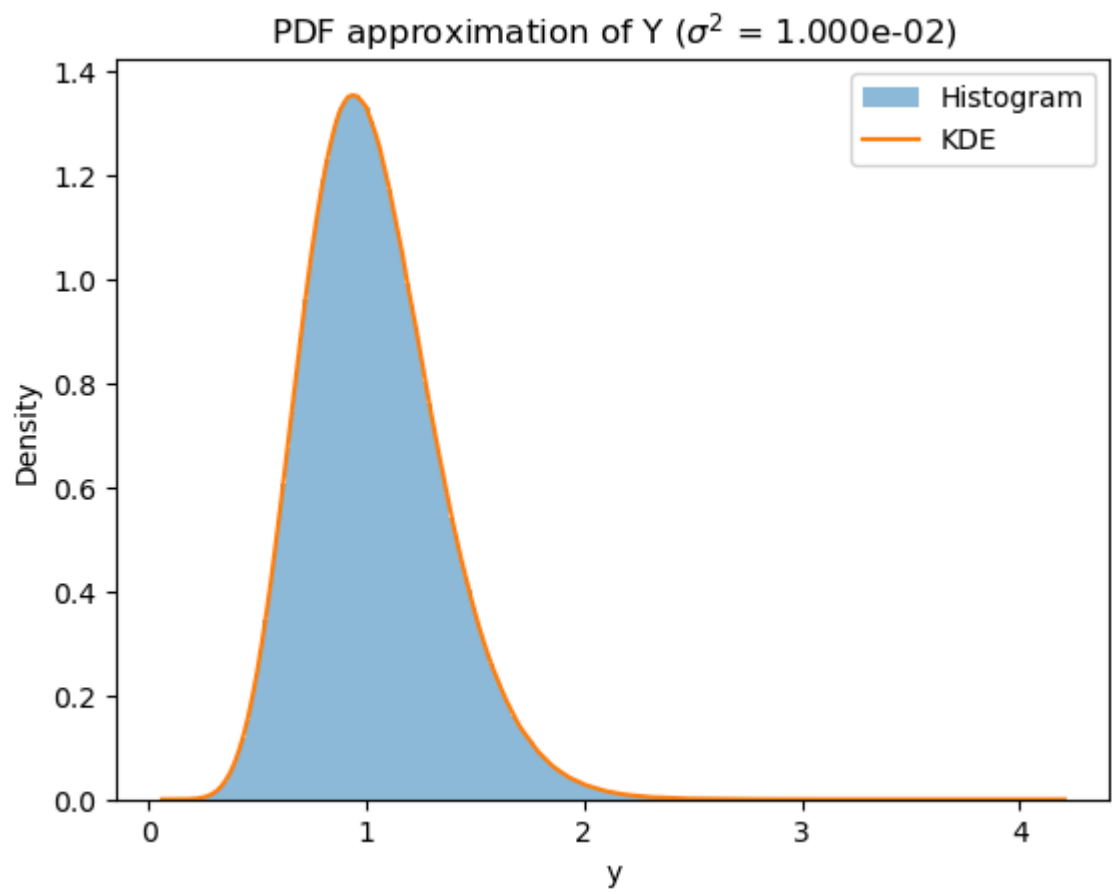
    print("Mean:", np.mean(Y))
    print("Variance:", np.var(Y))

    plt.figure()
    plt.hist(Y, bins=200, density=True, alpha=0.5, label="Histogram")
    sns.kdeplot(Y, label="KDE")

    plt.xlabel("y")
    plt.ylabel("Density")
    plt.legend()
    plt.title(f"PDF approximation of Y ( $\sigma^2 = \{{\sigma}^2:.3e\}$ ")
    plt.show()

M = 10_000_000
sigma2 = 0.01
sigma = np.sqrt(sigma2)
f(sigma, M)
```

```
Mean: 1.0300767966822306
Variance: 0.09366696825670462
```

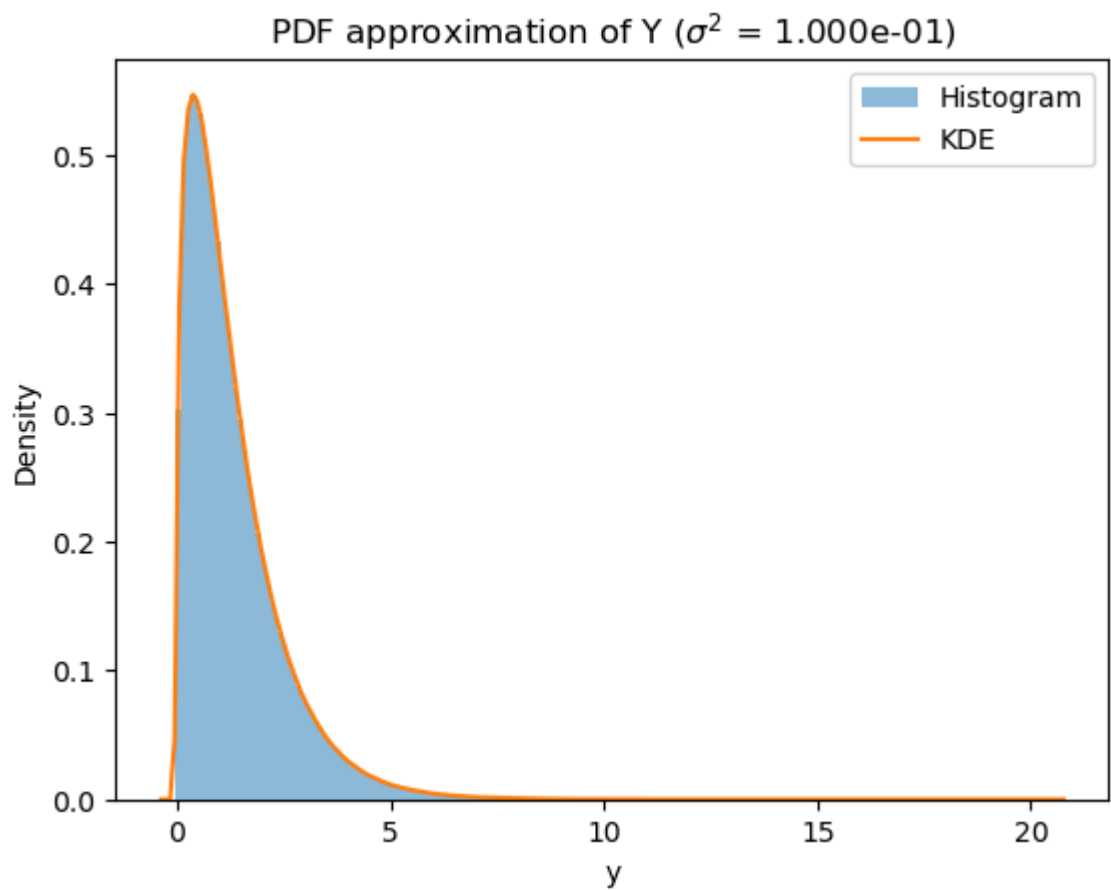


1.3

In [6]:

```
M = 10_000_000
sigma2 = 0.1
sigma = np.sqrt(sigma2)
f(sigma, M)
```

```
Mean: 1.2998122444060864
Variance: 1.2750207499557036
```



1.4

The perturbation method provides a good approximation when the input variance is small. For $\sigma^2 = 0.01$, the Monte Carlo estimates of the mean and variance are close to the first-order perturbation predictions, indicating that the linearization of the nonlinear function $f(X) = X^3$ around the mean is sufficiently accurate. However, for $\sigma^2 = 0.1$, the empirical mean and variance deviate significantly from the perturbation estimates. This occurs because the cubic transformation introduces strong nonlinear effects, and the first-order Taylor approximation is no longer adequate over the wider range of possible input values. This illustrates that perturbation methods are most reliable when the input uncertainty is small or when the function is approximately linear near the mean. For strongly nonlinear functions or larger uncertainties, higher-order expansions or sampling-based methods such as Monte Carlo simulation are required for accurate uncertainty propagation.

p2

Exported 3/12/2026, 4:36:39 PM

2.1

In [86]:

```
import numpy as np
import scipy.io as sio
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf

data = sio.loadmat('HW08_Problem2.mat')

print(data.keys())
```

```
dict_keys(['__header__', '__version__', '__globals__', 'q_samples',
'variance0_samps'])
```

In [87]:

```
q, s2 = data['q_samples'], data['variance0_samps']
qbar = np.mean(q, axis = 1)
print("Mean q, q_bar = ", qbar)
s2bar = np.mean(s2)
print("Mean variance, s2bar = ", s2bar)
```

```
Mean q, q_bar = [-1.73572551e+01  1.81973276e-03]
Mean variance, s2bar =  0.04200534049451376
```

2.2

In [88]:

```
import os, sys
sys.path.append('../hw1/')
import heat_rod as hr

x_target = 10.0 # cm

## ASSUMING ALL VALUES AS IN HW08 problem 3
a = 0.95 # cm
b = 0.95 # cm
k = 2.3 # W/cm·C
Tamb = 21.29 # C
L = 70
# Create the rod model
rod = hr.SteadyStateRod(a, b, None, k, None, Tamb, L)

# Evaluate Ts at x=10 for each MCMC sample
Ts_samples = np.zeros(len(q.T))

for i, (Phi, h) in enumerate(q.T):
    Ts_samples[i] = rod.Ts(x_target, h=h, k=k, Phi=Phi, Tamb=Tamb)

# Calculate empirical mean and variance
Ts_mean = np.mean(Ts_samples)
Ts_variance = np.var(Ts_samples)

print("\nProblem 2.2:")
print(f"At x = {x_target} cm:")
print(f"Empirical mean of Ts: {Ts_mean:.6f} °C")
print(f"Empirical variance of Ts: {Ts_variance:.6f}")
```

```
Problem 2.2:
At x = 10.0 cm:
Empirical mean of Ts: 94.790544 °C
Empirical variance of Ts: 0.383737
```

2.3

In [89]:

```

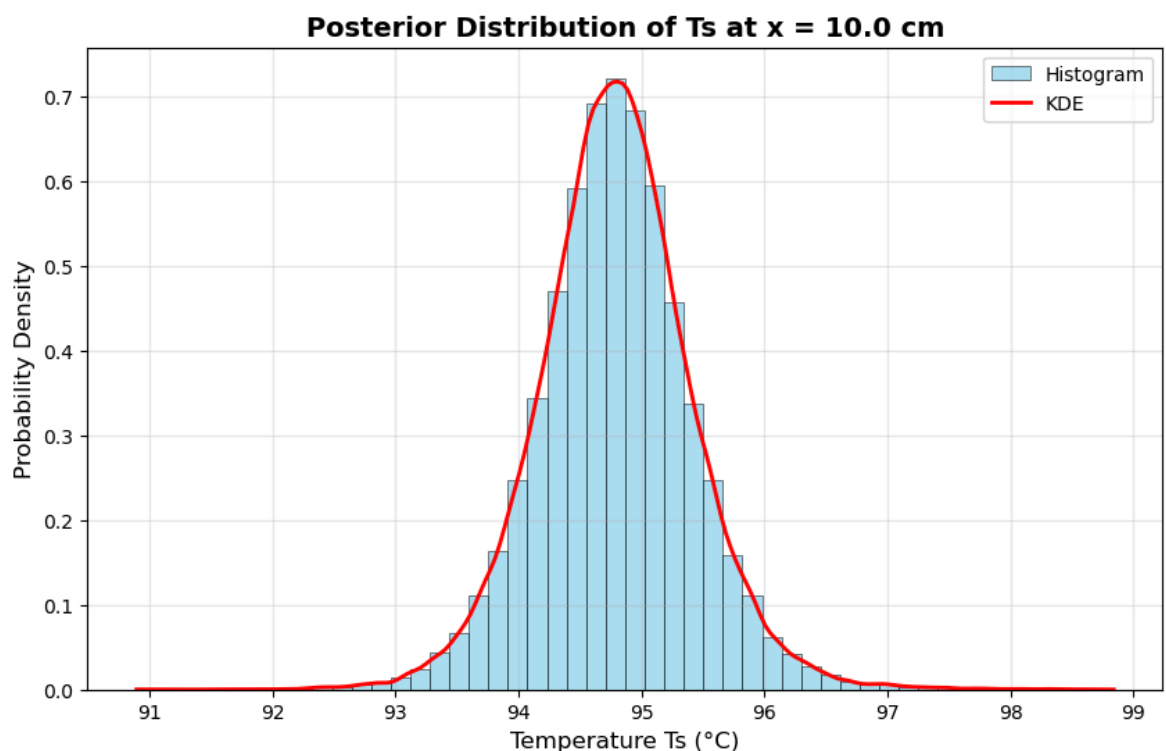
from scipy import stats
fig, ax = plt.subplots(figsize=(10, 6))

# Plot histogram
n_bins = 50
counts, bins, patches = ax.hist(Ts_samples, bins=n_bins, density=True,
                                alpha=0.7, color='skyblue',
                                edgecolor='black', linewidth=0.5,
                                label='Histogram')

kde = stats.gaussian_kde(Ts_samples)
x_range = np.linspace(Ts_samples.min(), Ts_samples.max(), 200)
kde_values = kde(x_range)
ax.plot(x_range, kde_values, 'r-', linewidth=2, label='KDE')

ax.set_xlabel('Temperature Ts (°C)', fontsize=12)
ax.set_ylabel('Probability Density', fontsize=12)
ax.set_title(f'Posterior Distribution of Ts at x = {x_target} cm',
             fontsize=14, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)

```



2.4

In [90]:

```

print("\nProblem 2.4:")
def ci(Ts_samples):
    sorted_Ts = np.sort(Ts_samples)
    n_samples = len(sorted_Ts)

    # Find indices for the 2.5th and 97.5th percentiles
    lower_idx = int(np.floor(0.025 * n_samples)) # 2.5% of samples
    upper_idx = int(np.ceil(0.975 * n_samples)) - 1 # 97.5% of
samples

    # Get the values at these indices
    lower_bound_manual = sorted_Ts[lower_idx]
    upper_bound_manual = sorted_Ts[upper_idx]

    print(f"\nManual calculation (order statistics):")
    print(f"Total samples: {n_samples}")
    print(f"Lower bound (index {lower_idx}): {lower_bound_manual:.4f}
°C")
    print(f"Upper bound (index {upper_idx}): {upper_bound_manual:.4f}
°C")

    # Verify that approximately 95% of samples fall within this
interval
    within_interval = np.sum((Ts_samples >= lower_bound_manual) &
(Ts_samples <= upper_bound_manual))
    percentage_within = (within_interval / n_samples) * 100

    print(f"\nVerification:")
    print(f"Samples within interval: {within_interval} out of
{n_samples} ({percentage_within:.2f}%)")

    # Visualize the credible interval
    import matplotlib.pyplot as plt

    fig, (ax1) = plt.subplots(1, 1, figsize=(14, 5))

    # Plot 1: Histogram with credible interval
    ax1.hist(Ts_samples, bins=50, density=True, alpha=0.7,
color='skyblue', edgecolor='black')
    ax1.axvline(lower_bound_manual, color='red', linestyle='--',
linewidth=2, label=f'2.5%: {lower_bound_manual:.2f}°C')
    ax1.axvline(upper_bound_manual, color='red', linestyle='--',
linewidth=2, label=f'97.5%: {upper_bound_manual:.2f}°C')
    ax1.axvline(np.mean(Ts_samples), color='black', linestyle='-',
linewidth=2, label=f'Mean: {np.mean(Ts_samples):.2f}°C')
    ax1.fill_betweenx([0, ax1.get_ylim()[1]], lower_bound_manual,
upper_bound_manual, alpha=0.2, color='red', label='95% CI')

```

```

ax1.set_xlabel('Temperature Ts (°C)')
ax1.set_xlabel('Temperature Ts (°C)')
ax1.set_ylabel('Probability Density')
ax1.set_title(f'95% Credible Interval for Ts at x = 10 cm')
ax1.legend()
ax1.grid(True, alpha=0.3)
ci(Ts_samples=Ts_samples)

```

Problem 2.4:

Manual calculation (order statistics):

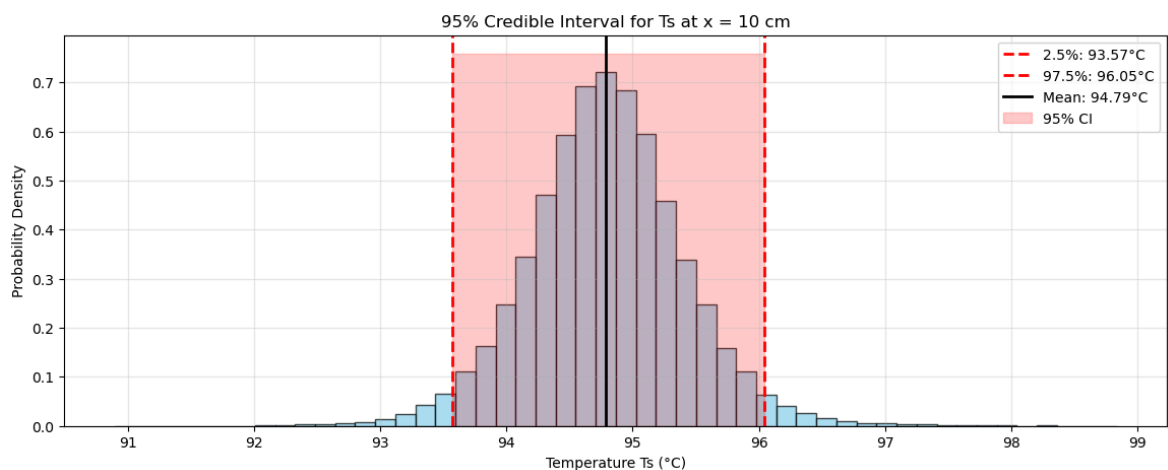
Total samples: 100000

Lower bound (index 2500): 93.5703 °C

Upper bound (index 97499): 96.0452 °C

Verification:

Samples within interval: 95001 out of 100000 (95.00%)



Interval: [93.5703 °C, 96.0452 °C]

2.5

In [91]:

```

alpha = 0.05 # for 95% prediction interval
def pi(Ts_samples = Ts_samples, s2 = s2):
    n_samples = len(Ts_samples)
    prediction_samples = np.zeros(n_samples)
    np.random.seed(42)
    s2 = s2.flatten()
    for i in range(n_samples):
        Ts_i = Ts_samples[i]

        sigma2_i = s2[i] #if s2.ndim == 1 else s2[i, 0]

        # Generate random error from N(0, sigma2_i)
        # Standard deviation is sqrt(sigma2_i)
        epsilon_i = np.random.normal(0, np.sqrt(sigma2_i))
        # Prediction sample = Ts + error
        prediction_samples[i] = Ts_i + epsilon_i
    return prediction_samples
prediction_samples = pi(Ts_samples)
ci(Ts_samples=prediction_samples)

```

Manual calculation (order statistics):

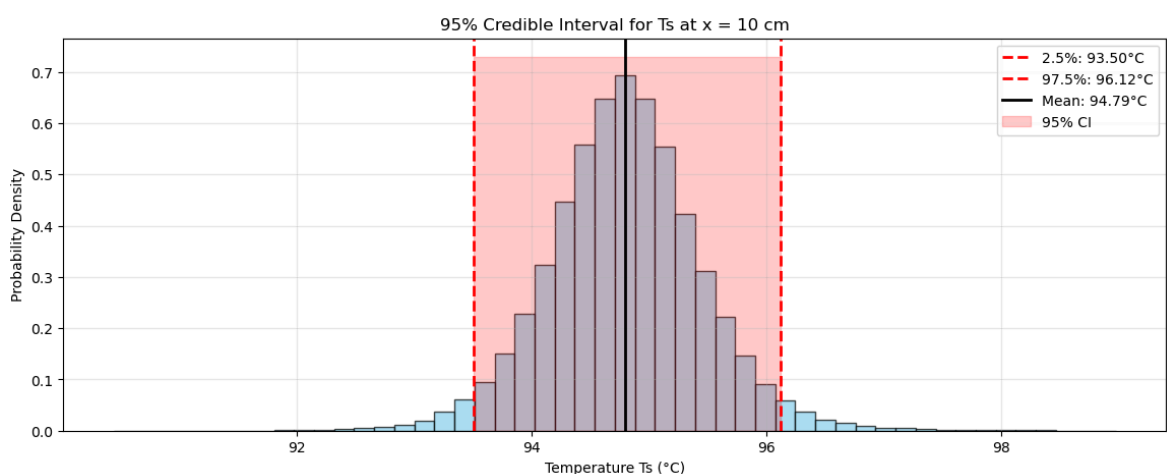
Total samples: 100000

Lower bound (index 2500): 93.4995 °C

Upper bound (index 97499): 96.1185 °C

Verification:

Samples within interval: 95000 out of 100000 (95.00%)



Interval: [93.4995 °C, 96.1185 °C]. Notice that the prediction interval is larger than the credibility interval.

2.6

In [94]:

```

x = np.array([
    10, 14, 18, 22, 26, 30, 34, 38,
    42, 46, 50, 54, 58, 62, 66
])

y_obs = np.array([
    96.14, 80.12, 67.66, 57.96, 50.90, 44.84, 39.75, 36.16,
    33.31, 31.15, 29.28, 27.88, 27.18, 26.40, 25.86
])

# Number of MCMC samples
n_samples = q.shape[1]

# Initialize arrays to store results for each x location
n_x = len(x)
Ts_mean_all = np.zeros(n_x)
Ts_lower_cred_all = np.zeros(n_x)
Ts_upper_cred_all = np.zeros(n_x)
Ts_lower_pred_all = np.zeros(n_x)
Ts_upper_pred_all = np.zeros(n_x)

# For each x location, compute credibility and prediction intervals
for idx, x_val in enumerate(x):
    print(f"Processing x = {x_val} cm...")

    # Evaluate Ts at this x for each MCMC sample
    Ts_samples_x = np.zeros(n_samples)
    for i in range(n_samples):
        Phi = q[0, i]
        h = q[1, i]
        Ts_samples_x[i] = rod.Ts(x_val, h=h, k=k, Phi=Phi, Tamb=Tamb)

    # Calculate mean Ts
    Ts_mean_all[idx] = np.mean(Ts_samples_x)

    # Calculate 95% credible interval (parameter uncertainty only)
    Ts_lower_cred_all[idx] = np.percentile(Ts_samples_x, 2.5)
    Ts_upper_cred_all[idx] = np.percentile(Ts_samples_x, 97.5)

    # Generate prediction samples (including measurement error)
    np.random.seed(42) # for reproducibility
    prediction_samples_x = np.zeros(n_samples)
    # for i in range(n_samples):
    #     # Add random error based on corresponding variance sample
    #     epsilon_i = np.random.normal(0, np.sqrt(s2[i]))
    #     print(epsilon_i)
    #     prediction_samples_x[i] = Ts_samples_x[i] + epsilon_i

```

```
epsilon = np.random.normal(0, np.sqrt(s2))
prediction_samples_x = Ts_samples_x + epsilon

# Calculate 95% prediction interval
Ts_lower_pred_all[idx] = np.percentile(prediction_samples_x, 2.5)
Ts_upper_pred_all[idx] = np.percentile(prediction_samples_x, 97.5)

# Calculate the difference between observations and predicted values
using mean q
# Using mean q (qbar) for deterministic prediction
qbar = np.mean(q, axis=1)
Phi_bar, h_bar = qbar[0], qbar[1]

# Calculate Ts_pred using mean parameters
Ts_pred_mean_params = np.zeros(n_x)
for idx, x_val in enumerate(x):
    Ts_pred_mean_params[idx] = rod.Ts(x_val, h=h_bar, k=k,
    Phi=Phi_bar, Tamb=Tamb)

# Calculate residuals: y_obs - Ts(x, qbar)
residuals = y_obs - Ts_pred_mean_params
```

```
Processing x = 10 cm...
Processing x = 14 cm...
Processing x = 18 cm...
Processing x = 22 cm...
Processing x = 26 cm...
Processing x = 30 cm...
Processing x = 34 cm...
Processing x = 38 cm...
Processing x = 42 cm...
Processing x = 46 cm...
Processing x = 50 cm...
Processing x = 54 cm...
Processing x = 58 cm...
Processing x = 62 cm...
Processing x = 66 cm...
```

In [99]:

```

import matplotlib.pyplot as plt

# Create figure
fig, (ax1) = plt.subplots(1, 1, figsize=(8,7), sharex=True)

# -----
# TOP PANEL: predictions + intervals
# -----

# Observations
# ax1.scatter(x, y_obs, color='black', label='Observations', marker =
"*)")

# Mean predicted temperature
# ax1.plot(x, Ts_mean_all, color='blue', label='Mean prediction')

# Credible interval (parameter uncertainty)
ax1.fill_between(
    x,
    Ts_lower_cred_all - Ts_pred_mean_params,
    Ts_upper_cred_all - Ts_pred_mean_params,
    color='blue',
    alpha=0.3,
    label='95% Credible interval'
)

cred_lower_err = Ts_mean_all - Ts_lower_cred_all
cred_upper_err = Ts_upper_cred_all - Ts_mean_all

# ax1.errorbar(
#     x,
#     Ts_mean_all,
#     yerr=[cred_lower_err, cred_upper_err],
#     fmt='none',          # no marker for the mean
#     ecolor='blue',
#     elinewidth=2,
#     capsize=5,
#     label='95% Credible interval'
# )

# Prediction interval (includes measurement noise)
ax1.fill_between(
    x,
    Ts_lower_pred_all - Ts_pred_mean_params,
    Ts_upper_pred_all - Ts_pred_mean_params,
    color='red',
    alpha=0.1,

```

```
        label='95% Prediction interval'
    )

    pred_lower_err = Ts_mean_all - Ts_lower_pred_all
    pred_upper_err = Ts_upper_pred_all - Ts_mean_all

    # ax1.errorbar(
    #     x,
    #     Ts_mean_all,
    #     yerr=[pred_lower_err, pred_upper_err],
    #     fmt='none',
    #     ecolor='red',
    #     elinewidth=1.5,
    #     capsize=4,
    #     label='95% Prediction interval'
    # )

    ax1.set_ylabel("Temperature (°C)")
    ax1.set_title("Posterior Predictions with Credible and Prediction
Intervals")
    ax1.legend()

    # -----
    # BOTTOM PANEL: residuals
    # -----

    # Residuals
    ax1.scatter(x, residuals, color='black', label='Residuals')

    # zero reference line
    ax1.axhline(0, color='k', linestyle='--')

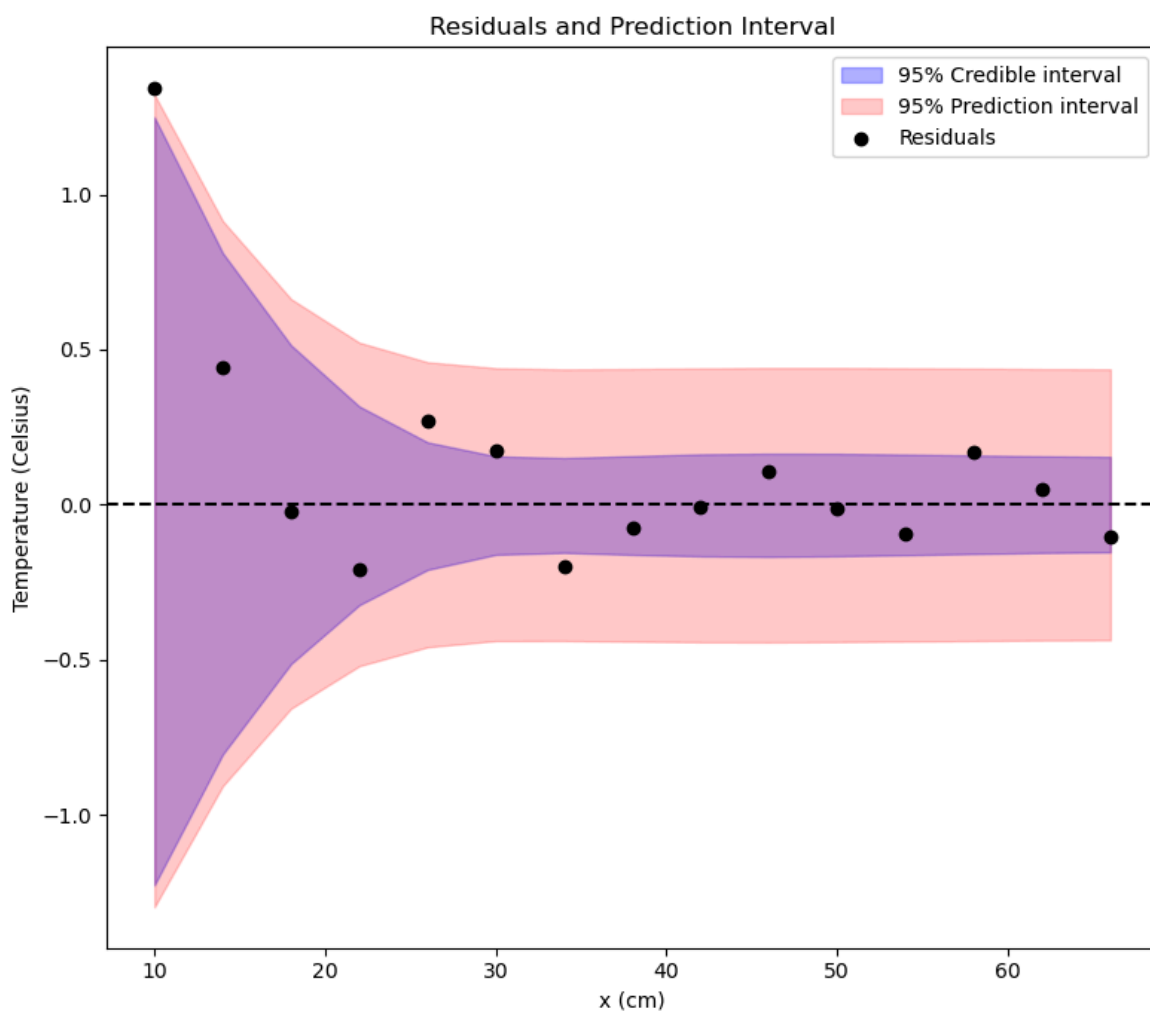
    # Prediction interval centered around zero
    lower_centered = Ts_lower_pred_all - Ts_pred_mean_params
    upper_centered = Ts_upper_pred_all - Ts_pred_mean_params

    # ax1.fill_between(
    #     x,
    #     lower_centered,
    #     upper_centered,
    #     color='green',
    #     alpha=0.2,
    #     label='Prediction interval'
    # )

    pred_lower_err = Ts_mean_all - Ts_lower_pred_all
    pred_upper_err = Ts_upper_pred_all - Ts_mean_all
```



```
# ax1.errorbar(  
#     x,  
#     residuals,  
#     yerr=[pred_lower_err, pred_upper_err],  
#     fmt='none',  
#     ecolor='red',  
#     elinewidth=1.5,  
#     capsize=4,  
#     label='95% Prediction interval'  
# )  
  
ax1.set_xlabel("x (cm)")  
ax1.set_ylabel(r"Temperature (Celsius)")  
ax1.set_title("Residuals and Prediction Interval")  
ax1.legend()  
  
plt.tight_layout()  
plt.show()
```



In [100]:

```
inside = (y_obs >= Ts_lower_pred_all) & (y_obs <= Ts_upper_pred_all)

percent_inside = 100 * np.sum(inside) / len(y_obs)

print(f"Percentage of observations inside prediction interval:
{percent_inside:.1f}%")
```

```
Percentage of observations inside prediction interval: 93.3%
```

We expect 95% of observations to lie within the prediction intervals. Empirically, we have 93.3% observations within the prediction intervals, which is nearly equal to the expected/correct percentage of observations within the prediction interval.

p3

Exported 3/12/2026, 4:39:11 PM

In []:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm, uniform
import sys
sys.path.append('../hw1/')
import heat_rod as hr

# Set random seed for reproducibility
np.random.seed(42)

# Problem parameters
k = 2.3 # W/cm/°C
Tamb = 21.29 # °C
a = 0.95 # cm
b = 0.95 # cm
L = 70 # cm
x_target = 38 # cm

# Distribution parameters for  $q = [\Phi, h]^T$ 
#  $\Phi \sim N(-15, 10)$  in W/cm2
phi_mean = -15
phi_var = 10
phi_std = np.sqrt(phi_var)

#  $h \sim U(5e-4, 0.0015)$  in W/cm2/°C
h_lower = 5e-4
h_upper = 0.0015

# Create the rod model
rod = hr.SteadyStateRod(a, b, None, k, None, Tamb, L)

print("="*60)
print("PROBLEM 3.1: Monte Carlo Analysis (Large Sample)")
print("="*60)

# Generate large number of samples for accurate estimate
N_large = 100000
phi_samples_large = np.random.normal(phi_mean, phi_std, N_large)
h_samples_large = np.random.uniform(h_lower, h_upper, N_large)

# Evaluate Ts at x=38 for each sample
Ts_samples_large = np.zeros(N_large)
for i in range(N_large[:]):
    Ts_samples_large[i] = rod.Ts(x_target, h=h_samples_large[i], k=k,
                                Phi=phi_samples_large[i], Tamb=Tamb)

# Calculate mean and variance

```

The documents are created with Code-Format: <https://code-format.com/>

```

Ts_mean_true = np.mean(Ts_samples_large)
Ts_var_true = np.var(Ts_samples_large)

# # Calculate standard errors
# std_error_mean = np.std(Ts_samples_large) / np.sqrt(N_large)
# std_error_var = np.sqrt(2/N_large) * Ts_var_true # Approximate for
normal distribution

print(f"Based on {N_large:}, MC samples:")
print(f" $\mathbb{E}[Ts]$  = {Ts_mean_true:.6f} °C")

print(f"\nvar[Ts] = {Ts_var_true:.6f} (°C)2")
print(f"std[Ts] = {Ts_var_true**.5:.6f} (°C)2")

```

```

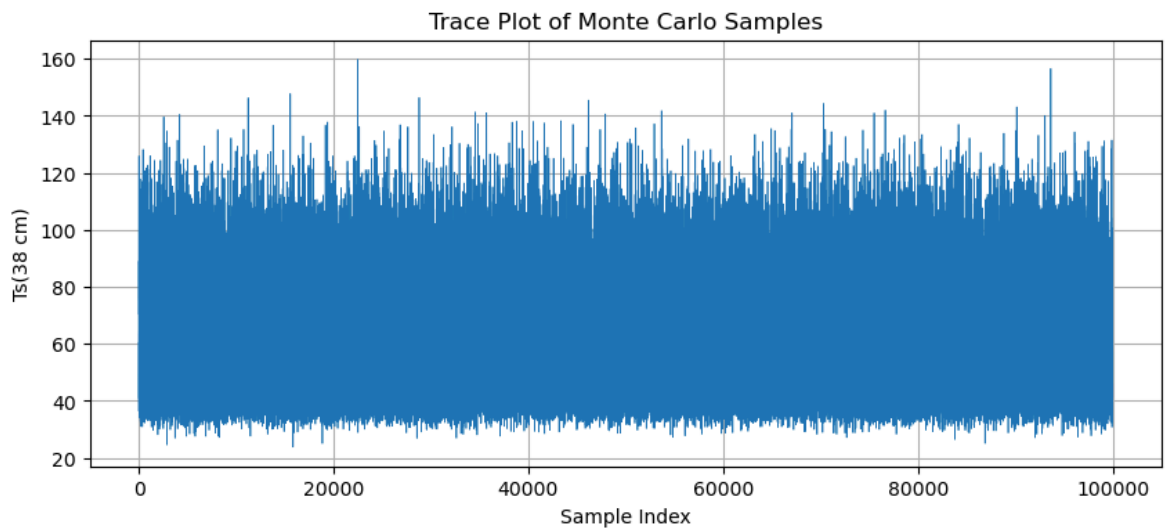
=====
PROBLEM 3.1: Monte Carlo Analysis (Large Sample)
=====
Based on 100,000 MC samples:
 $\mathbb{E}[Ts]$  = 57.977335 °C

var[Ts] = 336.988485 (°C)2
std[Ts] = 18.357246 (°C)2

```

In [57]:

```
plt.figure(figsize=(10,4))
plt.plot(Ts_samples_large, linewidth=0.5)
plt.xlabel("Sample Index")
plt.ylabel("Ts(38 cm)")
plt.title("Trace Plot of Monte Carlo Samples")
plt.grid()
plt.show()
```

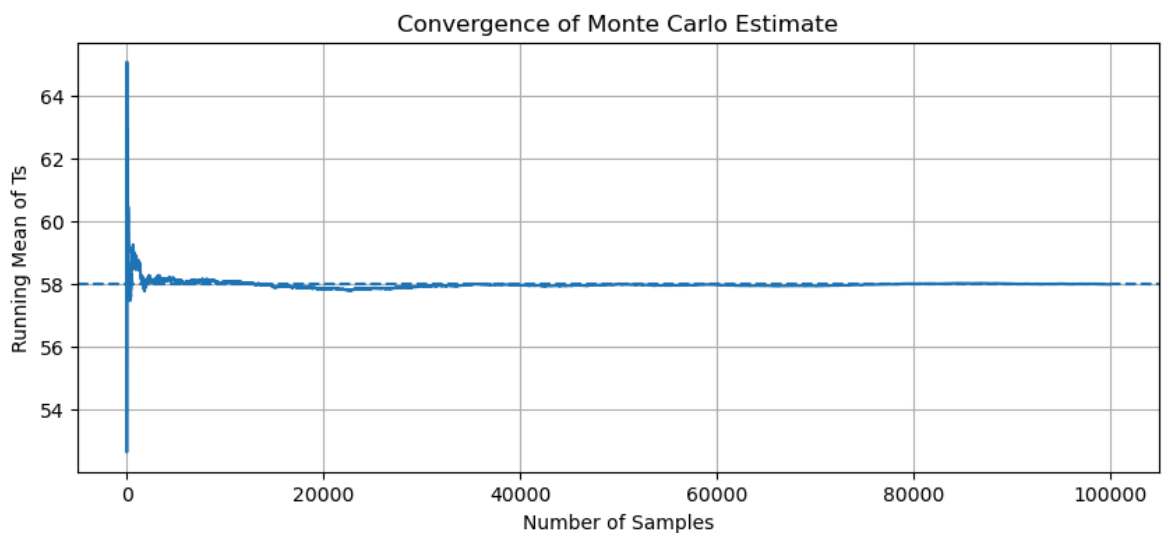


We have no visible drift and good sample spread across the temperature range. So, the MC algorithm is mixing well and our samples should well represent the posterior.

In [58]:

```
running_mean = np.cumsum(Ts_samples_large) / np.arange(1, N_large+1)

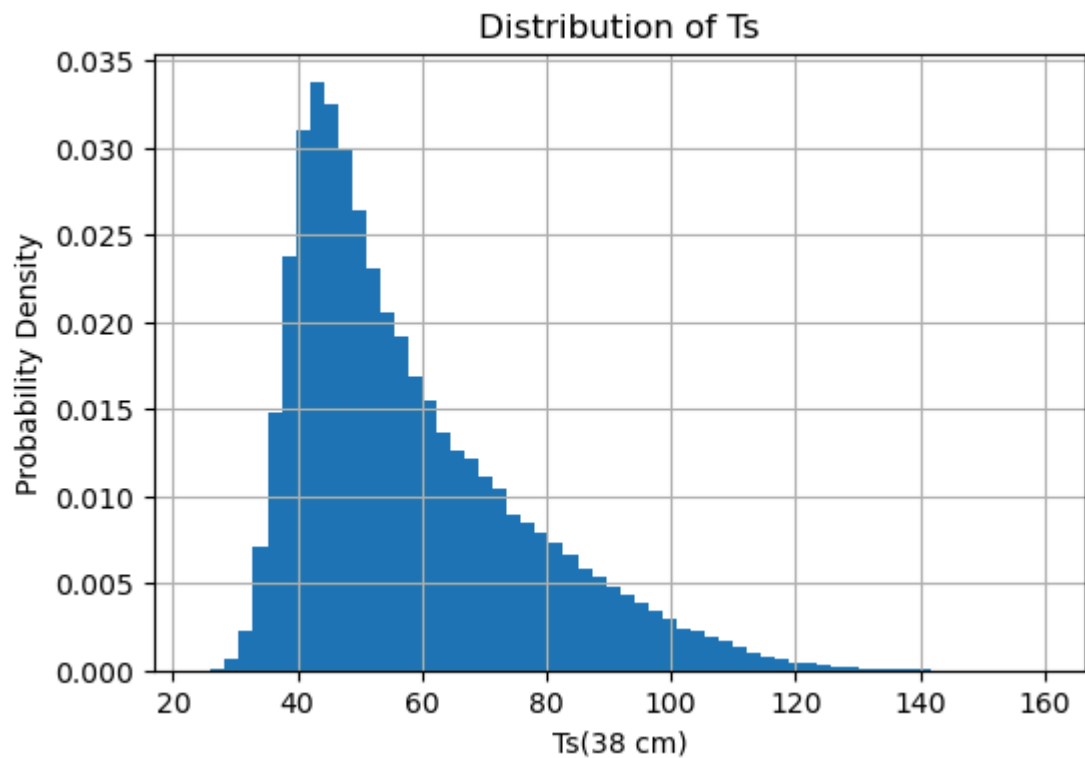
plt.figure(figsize=(10,4))
plt.plot(running_mean)
plt.axhline(Ts_mean_true, linestyle='--')
plt.xlabel("Number of Samples")
plt.ylabel("Running Mean of Ts")
plt.title("Convergence of Monte Carlo Estimate")
plt.grid()
plt.show()
```



Visually, the running mean stabilizes around 4000 samples, with not much noticeable change after 4000 samples. So, by 100000, the samples should generally have stabilized (the posterior has been well sampled). We thus take 4000 as a burnin period, and remove those samples.

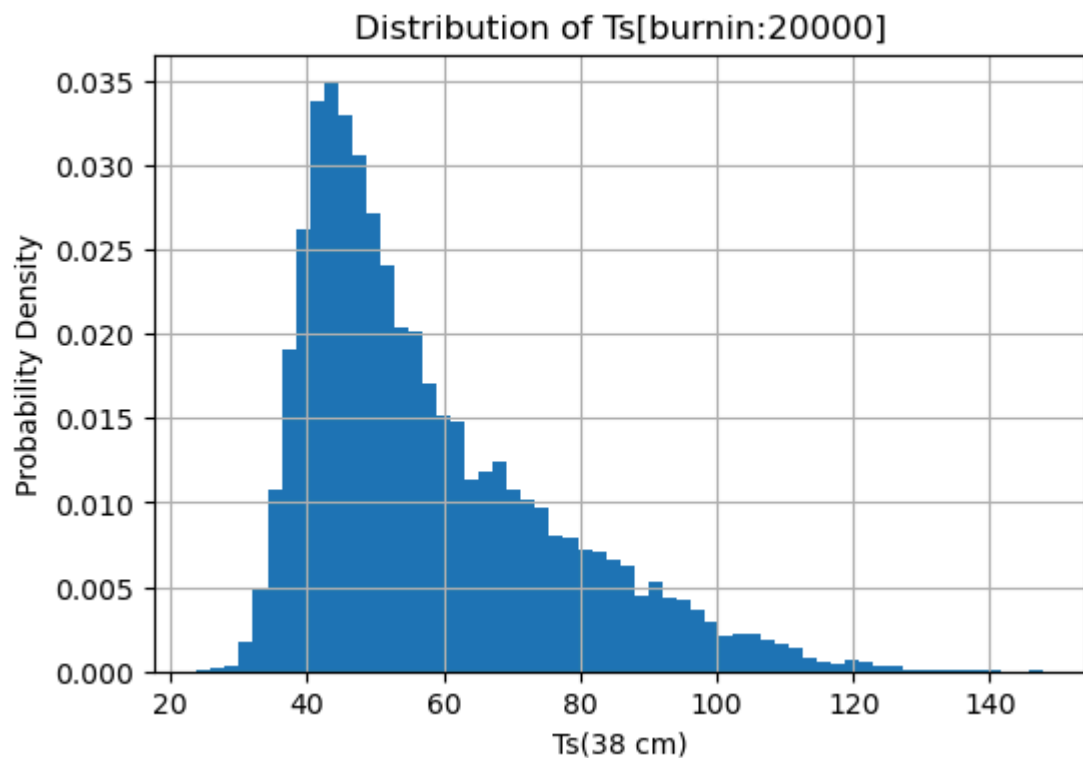
In [63]:

```
burnin = 4000
plt.figure(figsize=(6,4))
plt.hist(Ts_samples_large[burnin:], bins=60, density=True)
plt.xlabel("Ts(38 cm)")
plt.ylabel("Probability Density")
plt.title("Distribution of Ts")
plt.grid()
plt.show()
```



In [65]:

```
plt.figure(figsize=(6,4))
plt.hist(Ts_samples_large[burnin:20000], bins=60, density=True)
plt.xlabel("Ts(38 cm)")
plt.ylabel("Probability Density")
plt.title("Distribution of Ts[burnin:20000]")
plt.grid()
plt.show()
```



Visually, as mentioned above, after the burnin samples, there is minimal change to the posterior distribution. This is of course a qualitative description rather than a strong quantitative guarantee on the accuracy of our MC simulation.

In [74]:

```
Ts_mean_true = np.mean(Ts_samples_large[burnin:])
Ts_var_true = np.var(Ts_samples_large[burnin:])

print(f"Based on {N_large:,} MC samples:")
print(f" $\mathbb{E}[Ts]$  = {Ts_mean_true:.6f} °C")

print(f"\nvar[Ts] = {Ts_var_true:.6f} (°C)2")
print(f"std[Ts] = {Ts_var_true**.5:.6f} (°C)2")
```

Based on 100,000 MC samples:

$\mathbb{E}[Ts]$ = 57.972912 °C

var[Ts] = 336.684261 (°C)²

std[Ts] = 18.348958 (°C)²

A Monte Carlo simulation with $N=100000$ samples was performed, and we notice the running mean converges rapidly and stabilizes, demonstrating convergence of the estimator and the trace plot shows stationary fluctuations without drift, confirming the absence of bias or sampling instability. With the standard deviation of the mean, we have a quantitative and the standard error of the mean is only about 18, for a mean of about 56 C, also indicating reasonable convergence on the mean of the data.

3.2

In [75]:

```

def lhs_samples_explicit(N, phi_mean, phi_std, h_lower, h_upper, seed
= 42):
    """
    Generate Latin Hypercube Samples with explicit probability
    stratification
    """
    # Define the probability strata boundaries
    prob_bounds = np.linspace(0, 1, N+1) # [0, 1/N, 2/N, ..., 1]

    u_phi = np.zeros(N)
    u_h = np.zeros(N)

    for i in range(N):
        # Lower and upper bounds of this probability stratum
        p_low = prob_bounds[i]
        p_high = prob_bounds[i+1]

        # Sample uniformly within this probability stratum
        u_phi[i] = np.random.uniform(p_low, p_high)
        u_h[i] = np.random.uniform(p_low, p_high)

    # Shuffle to break correlation
    np.random.seed(seed) # for reproducibility
    np.random.shuffle(u_phi)
    np.random.shuffle(u_h)

    # Transform using inverse CDF
    phi_samples = norm.ppf(u_phi, loc=phi_mean, scale=phi_std)
    h_samples = h_lower + u_h * (h_upper - h_lower)

    return phi_samples, h_samples

# Demonstrate the difference
N_demo = 100
phi_mean, phi_std = -15, np.sqrt(10)
h_lower, h_upper = 5e-4, 0.0015

# Generate samples with corrected LHS
phi_lhs_correct, h_lhs_correct = lhs_samples_explicit(N_demo,
phi_mean, phi_std, h_lower, h_upper)

# Visualize the probability strata
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot showing probability strata for normal distribution
ax1 = axes[0]
x_phi = np.linspace(-25, 5, 1000)

```

```

cdf_phi = norm.cdf(x_phi, phi_mean, phi_std)

ax1.plot(x_phi, cdf_phi, 'b-', linewidth=2, label='CDF of  $\Phi$ ')
for i in range(N_demo+1):
    prob_i = i/N_demo
    if i < N_demo:
        # Find parameter value at this CDF level
        x_i = norm.ppf(prob_i, phi_mean, phi_std)
        ax1.axhline(y=prob_i, color='gray', linestyle='--', alpha=0.3)
        ax1.axvline(x=x_i, color='gray', linestyle='--', alpha=0.3)

    # Highlight the probability strata
    ax1.fill_betweenx([prob_i, (i+1)/N_demo],
                      norm.ppf(prob_i, phi_mean, phi_std),
                      norm.ppf((i+1)/N_demo, phi_mean, phi_std),
                      alpha=0.1, color='red')

ax1.set_xlabel('Φ (W/cm²)')
ax1.set_ylabel('Cumulative Probability')
ax1.set_title('Equal Probability Strata for Φ (Normal Distribution)')
ax1.grid(True, alpha=0.3)

# Plot showing probability strata for uniform distribution
ax2 = axes[1]
x_h = np.linspace(h_lower, h_upper, 1000)
cdf_h = (x_h - h_lower) / (h_upper - h_lower)

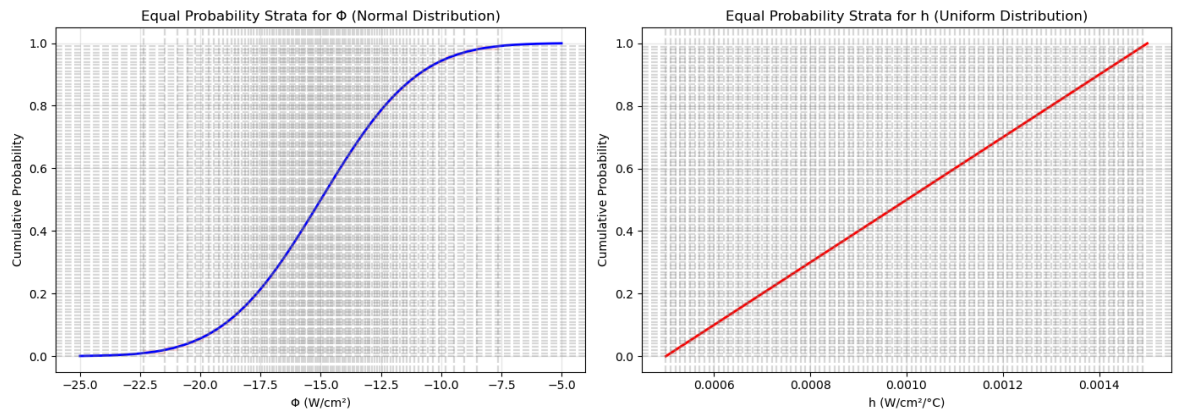
ax2.plot(x_h, cdf_h, 'r-', linewidth=2, label='CDF of h')
for i in range(N_demo+1):
    prob_i = i/N_demo
    if i < N_demo:
        # For uniform, parameter value is linear in probability
        x_i = h_lower + prob_i * (h_upper - h_lower)
        ax2.axhline(y=prob_i, color='gray', linestyle='--', alpha=0.3)
        ax2.axvline(x=x_i, color='gray', linestyle='--', alpha=0.3)

    # Highlight the probability strata
    ax2.fill_betweenx([prob_i, (i+1)/N_demo],
                      h_lower + prob_i * (h_upper - h_lower),
                      h_lower + ((i+1)/N_demo) * (h_upper -
h_lower),
                      alpha=0.1, color='red')

ax2.set_xlabel('h (W/cm²/°C)')
ax2.set_ylabel('Cumulative Probability')
ax2.set_title('Equal Probability Strata for h (Uniform Distribution)')
ax2.grid(True, alpha=0.3)

```

```
plt.tight_layout()  
plt.show()
```



In [76]:

```

print("\n" + "="*60)
print("PROBLEM 3.2: Comparison of MC vs LHS Sampling (N=100)")
print("="*60)

N = 100

# MC samples
phi_mc = np.random.normal(phi_mean, phi_std, N)
h_mc = np.random.uniform(h_lower, h_upper, N)

# LHS samples
phi_lhs, h_lhs = lhs_samples_explicit(N, phi_mean, phi_std, h_lower,
h_upper)

# Create side-by-side plots
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# MC samples plot
axes[0].scatter(phi_mc, h_mc, alpha=0.7, color='blue',
edgecolors='black')
axes[0].set_xlabel('Φ (W/cm²)', fontsize=12)
axes[0].set_ylabel('h (W/cm²/°C)', fontsize=12)
axes[0].set_title(f'Monte Carlo Samples (N={N})', fontsize=14)
# axes[0].grid(True, alpha=0.3)
axes[0].set_xlim([-25, -5]) # Adjust based on distribution
axes[0].set_ylim([h_lower - 0.0002, h_upper + 0.0002])

# LHS samples plot
axes[1].scatter(phi_lhs, h_lhs, alpha=0.7, color='red',
edgecolors='black')
axes[1].set_xlabel('Φ (W/cm²)', fontsize=12)
axes[1].set_ylabel('h (W/cm²/°C)', fontsize=12)
axes[1].set_title(f'Latin Hypercube Samples (N={N})', fontsize=14)
# axes[1].grid(True, alpha=0.3)
axes[1].set_xlim([-25, -5])
axes[1].set_ylim([h_lower - 0.0002, h_upper + 0.0002])

plt.tight_layout()
# Number of probability strata
num_boxes = N # smaller number so the grid is visible

# Vertical lines for Φ probability strata
for i in range(num_boxes + 1):
    p = i / num_boxes
    x_val = norm.ppf(p, phi_mean, phi_std)
    axes[0].axvline(x=x_val, color='gray', linestyle='-', alpha=0.4)

```

```

axes[1].axvline(x=x_val, color='gray', linestyle='-', alpha=0.4)

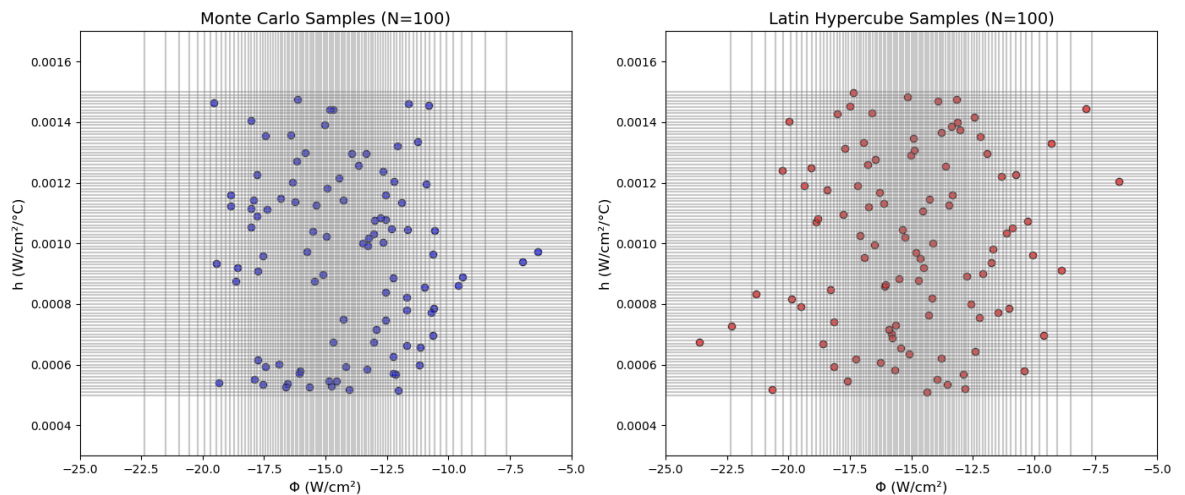
# Horizontal lines for h probability strata
for i in range(num_boxes + 1):
    p = i / num_boxes
    y_val = h_lower + p * (h_upper - h_lower)
    axes[0].axhline(y=y_val, color='gray', linestyle='-', alpha=0.4)
    axes[1].axhline(y=y_val, color='gray', linestyle='-', alpha=0.4)
plt.show()

```

```

=====
PROBLEM 3.2: Comparison of MC vs LHS Sampling (N=100)
=====

```



3.3

In [77]:

```

#
=====
=====
# PROBLEM 3.3: Histograms
#
=====
=====
print("\n" + "="*60)
print("PROBLEM 3.3: Histograms of Samples")
print("="*60)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

#  $\Phi$  histograms
axes[0, 0].hist(phi_mc, bins=15, density=True, alpha=0.7,
color='blue', edgecolor='black')
x_phi = np.linspace(-25, -5, 100)
axes[0, 0].plot(x_phi, norm.pdf(x_phi, phi_mean, phi_std), 'r-',
linewidth=2, label='True PDF')
axes[0, 0].set_xlabel('Φ (W/cm²)')
axes[0, 0].set_ylabel('Probability Density')
axes[0, 0].set_title(f'MC: Φ Distribution (N={N})')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].hist(phi_lhs, bins=15, density=True, alpha=0.7,
color='red', edgecolor='black')
axes[0, 1].plot(x_phi, norm.pdf(x_phi, phi_mean, phi_std), 'r-',
linewidth=2, label='True PDF')
axes[0, 1].set_xlabel('Φ (W/cm²)')
axes[0, 1].set_ylabel('Probability Density')
axes[0, 1].set_title(f'LHS: Φ Distribution (N={N})')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# h histograms
axes[1, 0].hist(h_mc, bins=15, density=True, alpha=0.7, color='blue',
edgecolor='black')
x_h = np.linspace(h_lower, h_upper, 100)
axes[1, 0].plot(x_h, uniform.pdf(x_h, h_lower, h_upper - h_lower), 'r-
', linewidth=2, label='True PDF')
axes[1, 0].set_xlabel('h (W/cm²/°C)')
axes[1, 0].set_ylabel('Probability Density')
axes[1, 0].set_title(f'MC: h Distribution (N={N})')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

```



```

axes[1, 1].hist(h_lhs, bins=15, density=True, alpha=0.7, color='red',
edgecolor='black')
axes[1, 1].plot(x_h, uniform.pdf(x_h, h_lower, h_upper - h_lower), 'r-
', linewidth=2, label='True PDF')
axes[1, 1].set_xlabel('h (W/cm2/°C)')
axes[1, 1].set_ylabel('Probability Density')
axes[1, 1].set_title(f'LHS: h Distribution (N={N})')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

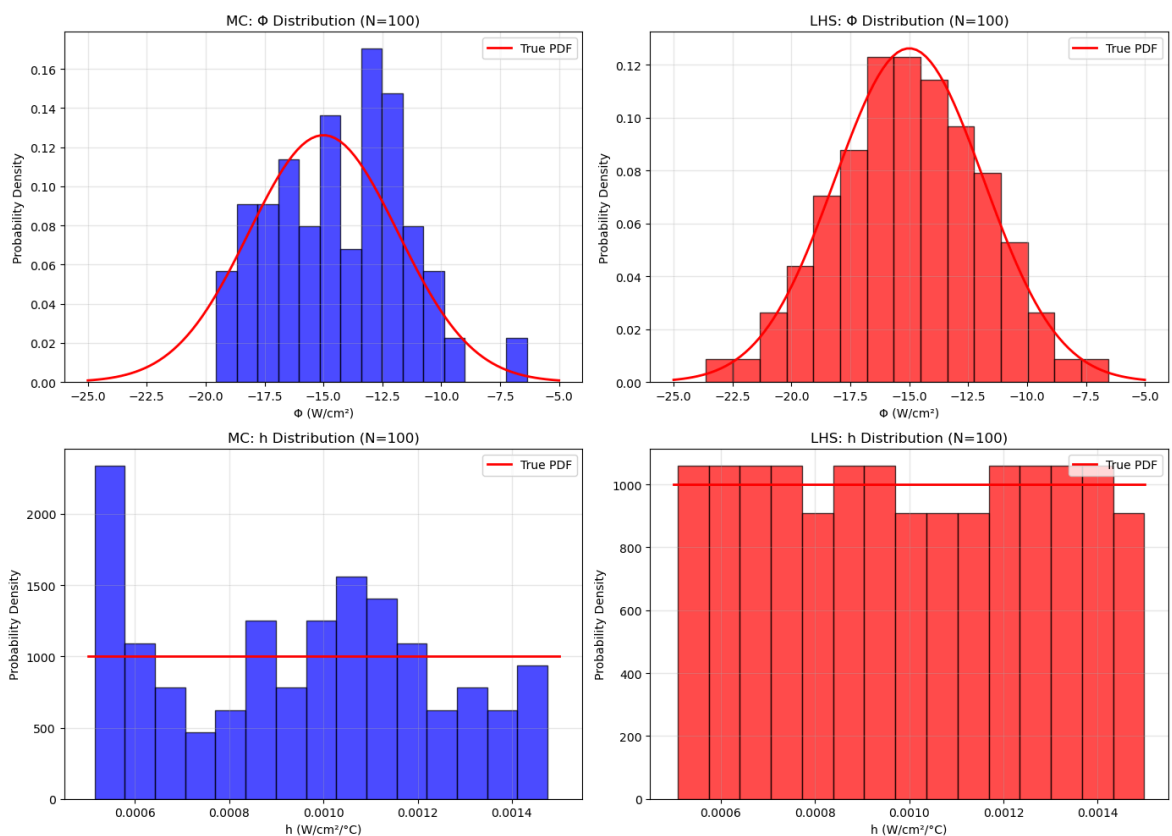
plt.tight_layout()
plt.show()

```

=====

PROBLEM 3.3: Histograms of Samples

=====



The histograms generated using Monte Carlo (MC) and Latin Hypercube Sampling (LHS) both approximate the underlying marginal distributions of the input variables ϕ and h . For ϕ , the histograms should resemble a normal distribution with mean -15 and variance 10 , while for h , they should approximate the uniform distribution on $[5 \times 10^{-4}, 1.5 \times 10^{-3}]$. However, because the sample size is relatively small ($N=100$), the Monte Carlo histograms often exhibit noticeable clustering and gaps. This occurs because purely random sampling may oversample some regions of the distribution while undersampling others.

In contrast, the LHS histograms tend to appear more evenly distributed across the support of each variable. This occurs because LHS stratifies the probability space into N equally probable intervals and forces exactly one sample to be drawn from each interval. As a result, LHS provides better coverage of the marginal distributions and reduces sampling variability compared to standard Monte Carlo sampling.

Overall, while both sampling methods approximate the correct distributions, LHS typically produces histograms that more closely resemble the true densities with fewer samples due to its stratified sampling strategy.

3.4 & 3.5

In [78]:

```

print("\n" + "="*60)
print("PROBLEMS 3.4 & 3.5: Estimates with N=100")
print("="*60)

# Calculate Ts for MC samples
Ts_mc = np.zeros(N)
for i in range(N):
    Ts_mc[i] = rod.Ts(x_target, h=h_mc[i], k=k, Phi=phi_mc[i],
Tamb=Tamb)

# Calculate Ts for LHS samples
Ts_lhs = np.zeros(N)
for i in range(N):
    Ts_lhs[i] = rod.Ts(x_target, h=h_lhs[i], k=k, Phi=phi_lhs[i],
Tamb=Tamb)

# MC estimates
Ts_mean_mc = np.mean(Ts_mc)
Ts_var_mc = np.var(Ts_mc)

# LHS estimates
Ts_mean_lhs = np.mean(Ts_lhs)
Ts_var_lhs = np.var(Ts_lhs)

print(f"Monte Carlo (N={N}):")
print(f"  E[Ts] = {Ts_mean_mc:.6f} °C")
print(f"  var[Ts] = {Ts_var_mc:.6f} (°C)²")

print(f"\nLatin Hypercube Sampling (N={N}):")
print(f"  E[Ts] = {Ts_mean_lhs:.6f} °C")
print(f"  var[Ts] = {Ts_var_lhs:.6f} (°C)²")

print(f"\nReference (N={N_large:,}):")
print(f"  E[Ts] = {Ts_mean_true:.6f} °C")
print(f"  var[Ts] = {Ts_var_true:.6f} (°C)²")

# Calculate errors
print(f"\nErrors relative to large-sample reference:")
print(f"  MC mean error: {abs(Ts_mean_mc - Ts_mean_true):.6f} °C
({abs(Ts_mean_mc - Ts_mean_true)/Ts_mean_true*100:.2f}%)")
print(f"  LHS mean error: {abs(Ts_mean_lhs - Ts_mean_true):.6f} °C
({abs(Ts_mean_lhs - Ts_mean_true)/Ts_mean_true*100:.2f}%)")
print(f"  MC var error: {abs(Ts_var_mc - Ts_var_true):.6f} (°C)²
({abs(Ts_var_mc - Ts_var_true)/Ts_var_true*100:.2f}%)")

```

```
print(f"  LHS var error: {abs(Ts_var_lhs - Ts_var_true):.6f} (°C)2
      ({abs(Ts_var_lhs - Ts_var_true)/Ts_var_true*100:.2f}%)" )
```

```
=====
PROBLEMS 3.4 & 3.5: Estimates with N=100
=====

Monte Carlo (N=100):
  E[Ts] = 58.963071 °C
  var[Ts] = 377.502056 (°C)2

Latin Hypercube Sampling (N=100):
  E[Ts] = 58.383613 °C
  var[Ts] = 366.118399 (°C)2

Reference (N=100,000):
  E[Ts] = 57.972912 °C
  var[Ts] = 336.684261 (°C)2

Errors relative to large-sample reference:
  MC mean error: 0.990159 °C (1.71%)
  LHS mean error: 0.410701 °C (0.71%)
  MC var error: 40.817795 (°C)2 (12.12%)
  LHS var error: 29.434139 (°C)2 (8.74%)
```

3.6

In [81]:

```

#
=====
=====
# PROBLEM 3.6: Comparison of MC vs LHS Performance
#
=====
=====

print("\n" + "="*60)
print("PROBLEM 3.6: MC vs LHS Performance Comparison")
print("="*60)

def evaluate_method(N, seed):
    """
    Run MC and LHS experiments for a given N and seed
    """
    np.random.seed(seed)

    # Monte Carlo samples
    phi_mc = np.random.normal(phi_mean, phi_std, N)
    h_mc = np.random.uniform(h_lower, h_upper, N)

    # LHS samples
    phi_lhs, h_lhs = lhs_samples_explicit(N, phi_mean, phi_std,
h_lower, h_upper, seed = seed)

    # Evaluate Ts
    Ts_mc = np.zeros(N)
    Ts_lhs = np.zeros(N)

    for i in range(N):
        Ts_mc[i] = rod.Ts(x_target, h=h_mc[i], k=k, Phi=phi_mc[i],
Tamb=Tamb)
        Ts_lhs[i] = rod.Ts(x_target, h=h_lhs[i], k=k, Phi=phi_lhs[i],
Tamb=Tamb)

    # Statistics
    mc_mean = np.mean(Ts_mc)
    mc_var = np.var(Ts_mc)

    lhs_mean = np.mean(Ts_lhs)
    lhs_var = np.var(Ts_lhs)

    return mc_mean, mc_var, lhs_mean, lhs_var

# Different sample sizes to test
N_values = [10, 50, 100, 200, 400, 600, 800]

```

```
# Different seeds
seeds = range(1,11)

# Store summary results
summary_results = []

for N_test in N_values:

    mc_means = []
    mc_vars = []

    lhs_means = []
    lhs_vars = []

    for seed in seeds:
        mc_mean, mc_var, lhs_mean, lhs_var = evaluate_method(N_test,
seed)

        mc_means.append(mc_mean)
        mc_vars.append(mc_var)

        lhs_means.append(lhs_mean)
        lhs_vars.append(lhs_var)

    # Compute averages across seeds
    mc_mean_of_means = np.mean(mc_means)
    mc_mean_of_vars = np.mean(mc_vars)
    mc_var_of_vars = np.var(mc_vars)

    lhs_mean_of_means = np.mean(lhs_means)
    lhs_mean_of_vars = np.mean(lhs_vars)
    lhs_var_of_vars = np.var(lhs_vars)

    summary_results.append([
        N_test,
        mc_mean_of_means,
        mc_mean_of_vars,
        mc_var_of_vars,
        lhs_mean_of_means,
        lhs_mean_of_vars,
        lhs_var_of_vars
    ])

# =====
# Print Summary Table
# =====
# The document was converted with Code-Format: https://code-format.com/
```

```
print(f"\nReference (N={N_large:,}):")
print(f"  E[Ts] = {Ts_mean_true:.6f} °C")
print(f"  var[Ts] = {Ts_var_true:.6f} (°C)²")

print("\n" + "="*70)
print("SUMMARY OF MC vs LHS PERFORMANCE (Averaged Over Seeds)")
print("="*70)

print(f"{'N':<8}{'MC mean':<15}{'MC var':<15}{'MC var variance':<15}
{'LHS mean':<15}{'LHS var':<15}{'LHS var variance'},{'MC mean error'},
{'LHS mean error'}")

for row in summary_results:
    merror = (row[1] - Ts_mean_true)/Ts_mean_true
    lhserror = (row[4] - Ts_mean_true)/ Ts_mean_true

    print(f"{row[0]:<8}{row[1]:<15.5f}{row[2]:<15.5f}{row[3]:<15.5f}
{row[4]:<15.5f}{row[5]:<15.5f}{row[6]:<15.5f}{merror:<15.5f}
{lhserror:<15.5f}")
```

```
=====
PROBLEM 3.6: MC vs LHS Performance Comparison
=====
```

Reference (N=100,000):

$\mathbb{E}[Ts] = 57.972912 \text{ }^{\circ}\text{C}$

$\text{var}[Ts] = 336.684261 \text{ } (^{\circ}\text{C})^2$

```
=====
SUMMARY OF MC vs LHS PERFORMANCE (Averaged Over Seeds)
=====
```

N	MC mean	MC var	MC var variance	LHS mean
LHS var	LHS var variance	MC mean error	LHS mean error	
10	58.41422	345.48698	51529.66629	57.99939
333.08348	10681.46977	0.00761	0.00046	
50	57.41362	327.95028	2565.41155	58.16888
347.04505	2956.49978	-0.00965	0.00338	
100	58.63580	359.54839	2652.89786	58.24108
354.19917	1113.90159	0.01143	0.00463	
200	57.84541	339.74196	694.38060	57.99705
326.01991	435.99043	-0.00220	0.00042	
400	57.95722	338.17556	427.46724	58.03837
336.12325	359.32074	-0.00027	0.00113	
600	57.68274	329.72017	354.52224	58.13164
342.57780	202.68871	-0.00501	0.00274	
800	57.80691	331.50598	557.65160	58.03981
333.75431	274.34255	-0.00286	0.00115	

var variance is the variance of the samples' variance, evaluated across seeds.

In [80]:

```
print(f"\nReference (N={N_large:,}):")
print(f"   $\mathbb{E}[Ts] = \{Ts\_mean\_true:.6f\} \text{ }^{\circ}\text{C}$ ")
print(f"   $\text{var}[Ts] = \{Ts\_var\_true:.6f\} \text{ } (^{\circ}\text{C})^2$ ")
```

Reference (N=100,000):

$\mathbb{E}[Ts] = 57.972912 \text{ }^{\circ}\text{C}$

$\text{var}[Ts] = 336.684261 \text{ } (^{\circ}\text{C})^2$

Latin Hypercube Sampling (LHS) is more efficient than standard Monte Carlo for small and moderate numbers of samples. It ensures better coverage of the input parameter space, producing estimates of the expectation and variance that are, in general, better representative of the input parameter space (as given by the reference large-sample MCMC method). For large N , both methods converge, but LHS still tends to have slightly reduced error (mean T_s vs true T_s) and variance (variance of T_s , as well as the variance across seeds/samples, var variance).